

02 · Instalación y ejecución

Documentacion del sistema

Version analizada: **0.3.0**

Commit analizado: **ff246ab**

Fecha de generacion: **2026-08-27**

Generado a partir de 02-installation-and-execution.md — los Markdown son la fuente unica.
No editar este PDF: editar el Markdown y regenerar.

Contenido de este documento

1. Requisitos previos
2. Opción A · Instalador de Windows (usuarios finales)
3. Opción B · Desde el código con `uv` (recomendada para desarrollo)
4. Opción C · Desde el código con `pip`
5. Comandos disponibles tras instalar
6. Ejecución del panel
7. Configuración inicial
8. Base de datos: no hay que crearla
9. Ejecución de las pruebas y de los validadores
10. Ejecución en producción
11. Compilación del binario
12. Errores frecuentes al instalar y arrancar
13. Verificación de que la instalación quedó bien

Cómo pasar de un clon limpio a un sistema funcionando, en cualquiera de las tres vías que el repositorio soporta. Todos los comandos de este documento se transcriben del propio repositorio; los que se ejecutaron durante el análisis llevan su salida real.

1. Requisitos previos

Requisito	Versión	Obligatorio	Evidencia
Python	<code>>=3.10</code>	Sí, salvo si se usa el instalador <code>.exe</code>	<code>pyproject.toml</code> , <code>requires-python</code>
Sistema operativo	Windows 10 / 11	Para el catálogo completo	Las acciones de UI, portapapeles, PowerShell y ventana activa son de Windows
<code>pip</code> o <code>uv</code>	—	Sí	<code>Makefile</code> ofrece ambos caminos
Chromium de Playwright	—	Solo para los 9 flows de navegador	<code>python -m playwright install chromium</code>
Binario <code>tesseract</code>	—	Solo para los flows con OCR (12, 17)	<code>plugins/analyzers/ocr_image_analyzer.py</code> lo busca y degrada si falta
Escritorio gráfico activo	—	Para capturas y acciones de teclado/ratón	<code>actions/screen.py</code> levanta <code>RuntimeError</code> sin escritorio

La CI valida **Python 3.10, 3.11 y 3.12** sobre `ubuntu-latest` y `windows-latest` (matriz de `.github/workflows/ci.yml`, seis combinaciones).

Nota sobre `playwright`. No está declarado en `pyproject.toml` ni en `requirements.txt`, pero nueve flows lo necesitan. Hay que instalarlo aparte. Las tres acciones que lo usan lo importan dentro de la función y devuelven un `RuntimeError` con el comando exacto, así que el fallo es legible; pero no se resuelve solo. Registrado en [15 · Riesgos y deuda técnica](#).

2. Opción A · Instalador de Windows (usuarios finales)

1. Descargar `Automa-Setup-vX.Y.Z.exe` del [último release](#).
2. Ejecutarlo. El asistente instala **por usuario** (`PrivilegesRequired=lowest` en `installer/Automa.iss`), sin pedir permisos de administrador.
3. Al terminar, «Launch Automa» abre una ventana nativa con el panel.

Detalles verificables del instalador, leídos de `installer/Automa.iss`:

Aspecto	Valor
Directorio por defecto	<code>{autopf}\Automa</code>
Idiomas	Español (por defecto) e inglés
Acceso directo en escritorio	Opcional, desmarcado por defecto (<code>Flags: unchecked</code>)
Arquitectura	<code>x64compatible</code>
Compresión	<code>lzma2/ultra</code> con <code>SolidCompression</code>
Desinstalación	Configuración → Aplicaciones → Automa

Efecto que sobrevive a la desinstalación. El binario empaquetado escribe sus datos fuera del directorio de instalación: `engine/paths.py::data_dir` devuelve `%LOCALAPPDATA%\Automa` cuando detecta que corre congelado. El desinstalador de Inno Setup borra `{app}`, no `%LOCALAPPDATA%`. **INFERENCIA:** la base de datos, los logs y las capturas **quedan en el disco tras desinstalar**. Si quiere borrarlos, hay que hacerlo a mano.

3. Opción B · Desde el código con `uv` (recomendada para desarrollo)

```
git clone https://github.com/vladimiracunadev-create/automa-pc.git
cd automa-pc
uv sync --extra dev --extra schema
python -m playwright install chromium      # necesario para los flows 02, 07 y 21-27
uv run automa-desktop                     # ventana nativa pywebview
```

`uv sync --extra dev --extra schema` es exactamente lo que hace la CI antes de correr los tests (`.github/workflows/ci.yml`, paso «Install project (runtime + dev + schema)»).

4. Opción C · Desde el código con `pip`

```
python -m venv .venv
.venv\Scripts\activate          # Windows
# source .venv/bin/activate      # Linux / macOS
pip install -e ".[dev,schema]"
python -m playwright install chromium
automa-desktop
```

Equivalente vía `Makefile`:

```
make install-dev    # pip install -e ".[dev,schema]"
make sync           # uv sync --extra dev --extra schema
```

`requirements.txt` **no basta**. Declara solo seis paquetes (`Pillow`, `psutil`, `requests`, `mss`, `pyautogui`, `pytesseract`) y le faltan `pyperclip`, `PyGetWindow` y `pywebview`, que sí están en `pyproject.toml`. `make install` usa ese archivo y deja el entorno incompleto: la ventana nativa y los flows 15 y 16 no arrancarían. Usar la vía `pyproject.toml` (opciones B o C). Registrado en [15 · Riesgos y deuda técnica](#).

5. Comandos disponibles tras instalar

Los cuatro *entry points* declarados en `pyproject.toml`, `[project.scripts]`:

Comando	Apunta a	Qué hace
<code>automa</code>	<code>engine.runner:main</code>	CLI con tres subcomandos: <code>list</code> , <code>run</code> , <code>scheduler</code>
<code>automa-panel</code>	<code>app.server:run_server</code>	Panel HTTP en <code>127.0.0.1:8787</code>
<code>automa-desktop</code>	<code>app.desktop:main</code>	Panel dentro de una ventana nativa <code>pywebview</code>
<code>automa-validate</code>	<code>scripts.validate_project:main</code>	Valida los 27 manifests contra el JSON Schema

Sintaxis del CLI

```
automa list                # inventario de flows en JSON
automa run flows/05_system_healthcheck  # ejecuta un flow
automa run flows/03_folder_inventory --context mi_contexto.json
automa scheduler --interval 2          # bucle del scheduler, chequeo cada 2 s
```

Equivalentes sin instalar el paquete:

```
python -m engine.runner list
python -m engine.runner run flows/05_system_healthcheck
python -m app.server                # panel; equivale a make run-panel
```

[!] El CLI falla al imprimir en una consola Windows por defecto

Verificado durante este análisis. `engine/runner.py::main` termina con `print(json.dumps(..., ensure_ascii=False, indent=2))`. Los manifests y los estados contienen caracteres no representables en `cp1252` (por ejemplo →, presente en varias descripciones de flow), así que la impresión revienta:

```
$ python -m engine.runner list
UnicodeEncodeError: 'charmap' codec can't encode character '\u2192' in position 6042:
character maps to <undefined>
```

Ocurre con `list` y con `run`. **Detalle crítico:** en `run` el flow se ejecuta completo y correctamente —la corrida queda persistida en SQLite y el reporte escrito— y el error salta *después*, solo al volcar el JSON a la consola. No se pierde trabajo, pero el código de salida es de error.

Solución inmediata:

```
# PowerShell
$env:PYTHONIOENCODING = "utf-8"; python -m engine.runner list

# Git Bash / cmd
set PYTHONIOENCODING=utf-8
python -m engine.runner list
```

Verificado: con `PYTHONIOENCODING=utf-8` el comando devuelve el JSON completo sin error. El panel HTTP no sufre este problema porque escribe bytes UTF-8 a un socket, no a la consola. Registrado como hallazgo en [15 · Riesgos y deuda técnica](#) y con guía en [14 · Solución de problemas](#).

6. Ejecución del panel

```
automa-panel                # o: python -m app.server
```

Salida esperada: **Panel disponible en `http://127.0.0.1:8787`.**

El servidor es `ThreadingHTTPServer` de la biblioteca estándar (`app/server.py::run_server`), ligado a `127.0.0.1` por defecto. **Importar `app.server` tiene efectos secundarios:** al cargarse el módulo se arranca el scheduler en un hilo de fondo, se inicializa la base de datos y se sincroniza el catálogo de flows.

```
# app/server.py – se ejecuta al importar el módulo, no al llamar run_server()
SCHEDULER = SchedulerService(loop_sleep_seconds=2.0)
SCHEDULER.start_in_background()
init_db()
sync_flows(list_flows())
```

Consecuencia práctica: cualquier proceso que importe `app.server` —incluidos los tests de `tests/test_panel_endpoints.py`— levanta un scheduler. Es una decisión implícita, no documentada en el repositorio.

Parámetros de la ventana nativa

```
automa-desktop --host 127.0.0.1 --port 8787 --width 1200 --height 800 --fullscreen
```

`app/desktop.py::launch` arranca el servidor en un hilo, espera hasta 8 segundos a que el puerto responda (`_wait_for_server`) y solo entonces abre la ventana. Si el puerto no responde, imprime el error y devuelve código 1 sin abrir ventana. Si falta `pywebview`, devuelve código 2 con la instrucción de instalación.

7. Configuración inicial

No hace falta configurar nada para arrancar: los 27 flows traen su `context.example.json` y funcionan con los valores por defecto. Los flows web apuntan por defecto a HTML locales de `data/web/`, de modo que la demo corre sin internet.

Para personalizar un flow hay tres vías, resueltas en este orden por `engine/loader.py::FlowLoader.load_context`:

- 1. `configs/<carpeta>.json` — lo que escribe el panel desde `/flow/<folder>/config`
- 2. `flows/<carpeta>/context.user.json` — override local, no versionado
- 3. `flows/<carpeta>/context.example.json` — valor por defecto que viene con el flow

La primera que exista gana; las demás se ignoran por completo (no hay mezcla de claves). Detalle en [10 · Configuración](#).

Variables de entorno

Variable	Obligatoria	Efecto
<code>AUTOMA_PANEL_TOKEN</code>	No	Si se define, toda mutación del panel exige el header <code>X-Automa-Token</code> con ese valor
<code>AUTOMA_WEBHOOK_TOKEN</code>	No	Habilita <code>POST /api/hook/<folder></code> . Sin ella el webhook responde 401 siempre
<code>AUTOMA_ROOT</code>	No	Sobrescribe la raíz de solo lectura (donde viven <code>flows/</code> y <code>schemas/</code>)
<code>AUTOMA_DATA_ROOT</code>	No	Sobrescribe la raíz escribible (<code>db/</code> , <code>state/</code> , <code>logs/</code> , <code>configs/</code> , <code>secrets/</code>)
<code>PYTHONIOENCODING</code>	No	Recomendada en <code>utf-8</code> para usar el CLI en Windows (\$5)

Ninguna es obligatoria. Catálogo completo en [10 · Configuración](#).

8. Base de datos: no hay que crearla

`db/runs.db` se crea sola. `engine/database.py::init_db` ejecuta un `CREATE TABLE IF NOT EXISTS` de las siete tablas y se invoca desde `Orchestrator.__init__`, desde `SchedulerService.__init__`, desde el CLI y al importar `app.server`. No hay migraciones que aplicar ni semillas que cargar.

La única migración existente es una línea al final de `init_db`, que añade la columna `cron_expression` a bases anteriores a la v0.2.0:

```
_ensure_column(conn, 'schedules', 'cron_expression', 'cron_expression TEXT')
```

Detalle del esquema en [07 · Base de datos](#).

9. Ejecución de las pruebas y de los validadores

Los tres comandos que el README declara obligatorios antes de subir. **Salidas reales obtenidas en el commit analizado**, sobre Python 3.12.9 y Windows 11:

```
python -m pytest

150 passed in 16.62s
Required test coverage of 54% reached. Total coverage: 58.92%

python -m ruff check .

All checks passed!
```

```
python scripts/validate_project.py
```

```
{
  "ok": true,
  "flows_checked": 27,
  "registered_actions": 36,
  "errors": []
}
```

Y el smoke test de integración, que la CI corre en un job aparte:

```
python scripts/smoke_test.py
```

```
{
  "ok": true,
  "runs": 20,
  "db_path": "C:\\dev\\automa-pc\\db\\runs.db"
}
```

Efecto no obvio del smoke test. `scripts/smoke_test.py` llama a `set_flow_config('03_folder_inventory', {'path_override': 'data/inbox'})`, que **reescribe el archivo versionado** `configs/03_folder_inventory.json`. Tras ejecutarlo, `git status --porcelain` puede marcar ese archivo como modificado. Se restaura con `git checkout -- configs/03_folder_inventory.json`. Registrado en [15 · Riesgos y deuda técnica](#).

Atajos equivalentes del Makefile: `make test`, `make lint`, `make validate`, `make smoke`, `make flow-health`, `make list`, `make run-panel`, `make clean`.

10. Ejecución en producción

No hay «producción» en el sentido de servidor: el modo de uso previsto es el equipo del operador. Las dos formas soportadas son:

- **Instalador .exe** (opción A). Es el modo de distribución oficial.
- **automa-desktop desde el código** (opciones B y C).

Para dejar el scheduler corriendo sin ventana:

```
automa scheduler --interval 2
```

Ese proceso solo dispara las tareas programadas; no levanta panel. El panel arranca su propio scheduler al importarse, así que **no hay que ejecutar ambos**: dos schedulers sobre la misma base compiten, aunque la tabla `run_locks` impide que un mismo flow corra dos veces en paralelo (`engine/database.py:acquire_run_lock` usa la clave primaria `folder` y captura `sqlite3.IntegrityError`).

11. Compilación del binario

```
pwsh installer/build_local.ps1      # genera dist/Automa/
iscc installer/Automa.iss            # genera installer/output/Automa-Setup-vX.Y.Z.exe
```

Requiere PyInstaller e Inno Setup 6+. La CI lo hace en `windows-latest` ([.github/workflows/release.yml](#)), donde Inno Setup viene preinstalado como `iscc`. Detalle en [13 · Despliegue y operación](#).

REQUIERE VALIDACIÓN: el build no se ejecutó en este análisis. Hay un hallazgo abierto sobre `installer/automa.spec` (falta `actions.browser_extract` en `hiddenimports`) que solo se puede confirmar compilando.

12. Errores frecuentes al instalar y arrancar

Síntoma	Causa	Solución
<code>UnicodeEncodeError: 'charmap' codec...</code> al usar el CLI	Consola Windows en <code>cp1252</code> y <code>ensure_ascii=False</code>	<code>set PYTHONIOENCODING=utf-8</code> antes del comando (§5)

RuntimeError: playwright no está instalado	Flows 02, 07, 21–27 sin Playwright	<code>pip install playwright && python -m playwright install chromium</code>
ERROR: pywebview no esta instalado (código 2)	Falta <code>pywebview</code>	<code>pip install pywebview</code> , o usar <code>automa-panel</code> en el navegador
El flow OCR devuelve <code>status: "unavailable"</code>	Falta el binario <code>tesseract</code>	Windows: <code>choco install tesseract</code> . Linux: <code>apt-get install tesseract-ocr</code> . macOS: <code>brew install tesseract</code>
RuntimeError: No fue posible capturar la pantalla	Sin escritorio gráfico (sesión SSH, contenedor)	Ejecutar en una sesión Windows interactiva
PermissionError [WinError 5] al escribir en db/	Bundle instalado bajo <code>Program Files</code> con <code>AUTOMA_DATA_ROOT</code> mal apuntado	Corregido en v0.2.0 con <code>data_dir()</code> ; ver <code>CHANGELOG.md</code>
El puerto 8787 está ocupado	Otra instancia corriendo	<code>automa-desktop --port 8888</code> , o cerrar la otra instancia
Los 12 primeros atajos <code>Alt+N</code> funcionan, los demás no	El panel solo mapea 12 atajos	Ejecutar el flow haciendo clic en su tarjeta

Guía completa con diagnóstico y riesgo de cada solución en [14 · Solución de problemas](#).

13. Verificación de que la instalación quedó bien

```
python scripts/validate_project.py      # debe devolver "ok": true, 27 flows
curl http://127.0.0.1:8787/healthz     # debe devolver {"status": "ok"}
python -m engine.runner run flows/05_system_healthcheck # con PYTHONIOENCODING=utf-8
```

Si el tercero deja un archivo nuevo en `output/reports/` y una fila nueva en la pestaña **Histórico** del panel, el sistema está operativo de extremo a extremo.

Documentos relacionados: [01 · Descripción general](#) · [10 · Configuración](#) · [13 · Despliegue y operación](#) · [14 · Solución de problemas](#) · [18 · Guía para nuevo desarrollador](#)